



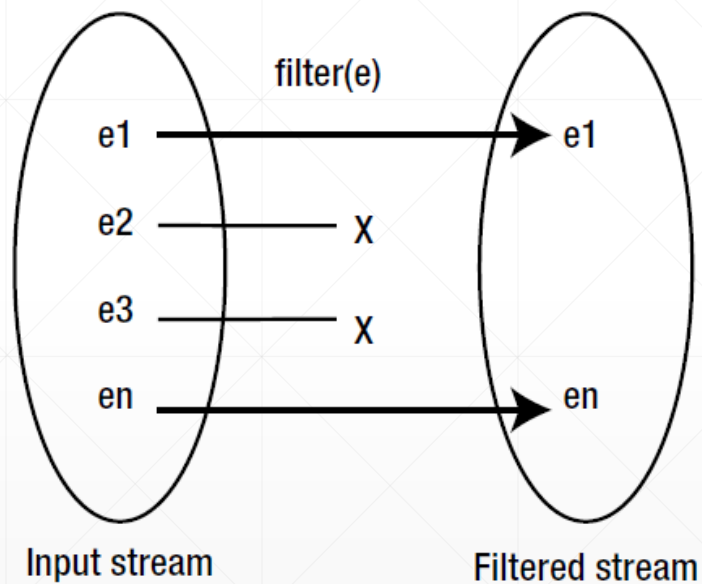
流中数据的过滤与查找

北京理工大学计算机学院
金旭亮

过滤



`filter()`: 接受一个 `Predicate`, 从流中排除某些元素。



```
static void useFilter() {  
    var nums : List<Integer> = List.of(1, 2, 3, 4, 5, 6);  
    //过滤出所有奇数并输出  
    nums.stream().filter(num -> num % 2 == 1)  
        .forEach(System.out::println);  
}
```

过滤可以很灵活

```
static void useFilter2() {  
    //过滤出所有女性员工  
    Person.persons().stream()  
        .filter(Person::isFemale)  
        .forEach(System.out::println);  
    System.out.println();  
    //级联filter, 过滤出男性且收入大于5000的员工  
    Person.persons().stream()  
        .filter(Person::isMale)  
        .filter(p -> p.getIncome() > 5000.0)  
        .forEach(System.out::println);  
    System.out.println();  
    //使用复合逻辑表达式, 完成同样的过滤功能  
    Person.persons().stream()  
        .filter(p -> p.isMale() && p.getIncome() > 5000.0)  
        .forEach(System.out::println);  
}
```

左边示例展示出了几种典型的实现过滤的方法。

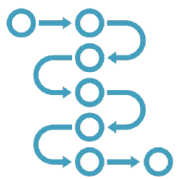
消除重复项

```
static void useDistinct(){  
    var numbers : List<Integer> =  
        Arrays.asList(1, 2, 1, 3, 3, 2, 4);  
    numbers.stream()  
        .distinct()  
        .forEach(System.out::println);  
}
```



List消除重复项的另一种方法，是将它放到Set中。Stream API提供有一个toSet收集器，可以完成这个任务。

排序



使用`sorted()`方法进行排序，需要向它提供一个比较器对象：

```
static void sortStream() {  
    List<String> words = List.of(  
        "apple", "oranage", "cat", "monkey");  
    //按照单词的长度进行排序  
    Stream<String> sortedWords =  
        words.stream().sorted(  
            Comparator.comparing(String::length));  
    sortedWords.forEach(System.out::println);  
}
```

查找单个元素

```
static Integer[] numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

```
static void findAny(Integer[] numbers) {  
    //findAny的返回值不是确定的 (尤其是在多线程环境下)  
    var anyResult : Optional<Integer> = Arrays.stream(numbers)  
        .filter(n -> n < 10)  
        .findAny();  
    //输出: 1  
    anyResult.ifPresent(System.out::println);  
}
```

可以使用Stream API所提供的findAny和findFirst函数查找符合条件的单个元素。

```
static void findFirst(Integer[] numbers) {  
    Predicate<Integer> lessThan10 = n -> n < 10;  
    //对于有序集合, findFirst结果是确定的  
    var firstResult : Optional<Integer> = Arrays.stream(numbers)  
        .filter(lessThan10)  
        .findFirst();  
    //输出: 1  
    firstResult.ifPresent(System.out::println);  
}
```

"满足"还是"不满足"?

```
static void useAllMatch() {  
    var words : List<String> = List.of("One", "Two",  
        "Three", "Four");  
    //集合中的所有元素, 是否都满足特定的约束条件?  
    var result : boolean = words.stream()  
        .allMatch(word -> word.length() >= 3);  
    System.out.println(result);  
}
```

使用Stream API提供的
allMatch和noneMath方法,
检查集合中的元素是否满足
或不满足特定的条件。

```
static void useNoneMatch() {  
    var words : List<String> = List.of("One", "Two",  
        "Three", "Four");  
    //是否集合中的所有元素, 都不以"t"结尾?  
    var result : boolean = words.stream()  
        .noneMatch(word -> word.endsWith("t"));  
    System.out.println(result);  
}
```

anyMatch方法

anyMatch 方法可以回答“流中是否至少有一个元素能匹配给定的条件”。

```
static void useAnyMatch() {  
    var words : List<String> = List.of("One", "Two",  
        "Three", "Four");  
    //集合中是否至少有一个单词以F打头?  
    var result : boolean = words.stream()  
        .anyMatch(word -> word.startsWith("F"));  
    System.out.println(result);  
}
```


切片操作: takeWhile



`takeWhile` 操作可以帮助你基于某个表达式对流进行分片（即便你要处理的流是无限流也毫无困难）。它会在遭遇第一个不符合要求的元素时停止处理。

```
static void useTakeWhile() {  
    var nums : List<Integer> = List.of(1, 2, 3, 4, 5, 6);  
    //取出小于6的第1个偶数  
    nums.stream()  
        .takeWhile(num -> {  
            System.out.println("takeWhile处理: " + num);  
            return num < 6 && num % 2 != 0;  
        })  
        .forEach(System.out::println);  
}
```

切片操作: dropWhile



`dropWhile` 操作是对 `takeWhile` 操作的补充。它会从头开始, 丢弃所有表达式结果为 `false` 的元素。一旦遭遇谓词计算的结果为 `true`, 它就停止处理, 并返回所有剩余的元素:

```
static void useDropWhile(){  
    List<Integer> numbers = List.of(1, 2, 1, 3, 3, 2, 4);  
    //删除集合前部所有小于3的元素, 然后返回剩余的所有元素  
    numbers.stream()  
        .dropWhile(num->num<3)  
        .forEach(System.out::println);  
}
```

流的短路求值



”

有些操作不需要处理全部数据就能得到结果。例如，假设需要对一个用`and`运算符连起来的包容多个布尔子表达式的大的布尔表达式求值。不管表达式有多长，只需发现其中有一个子表达式为`false`，就可以推断整个大的表达式将返回 `false`，用不着把所有子表达式全部都计算一遍，这就是“**短路**”。

对于流而言，某些操作（例如 `findFirst` 和 `findAny`）不用处理整个流，只要找到一个元素，就可以有结果了，它们就是“短路操作”。

同样，`limit`也是一个短路操作——它只需要创建一个给定大小的流，也用不着处理流中所有的元素。对于无限流，`limit`操作把无限流变成有限流。